Robustness to Paraphrasing. The LSH ensures that semantically equivalent paraphrases map to the same hash with high probability, as long as the embeddings remain within a certain proximity. This guarantees that the system tolerates adversarial paraphrasing, provided the changes do not cause significant shifts in the embedding space.

Advantages. This approach offers several benefits:

- **Robustness:** The use of LSH and rejection sampling ensures resilience to paraphrasing, aligning with the strong robustness definition.
- **Flexibility:** The bit-length l of the hash can be adjusted to balance efficiency and the reliability of the chosen LSH.
- **Scalability:** The rejection sampling process ensures that any n-bit message can be encoded reliably, regardless of its length.

Challenges. The main challenges of this approach lie in constructing LSH functions with suitable accuracy and the potential computational cost of rejection sampling.

Proofs. Proofs are given in Appendix C.

7 Implementation and Measurement

In this section, we detail our publicly available reference implementation of the two proposed steganographic systems: the watermarking-based scheme (for weak robustness) and the embedding-based scheme (for strong robustness). Both systems are fully open-source and available at https://github.com/NeilAPerry/robust_steganography. The code is designed to be modular, allowing developers to:

- Integrate new covertext sources (e.g., GPT-2, custom Hugging Face transformers), new perturbing functions for sampled tokens, and new PRFs for watermarking.
- Integrate new embedding models, new covertext sources, new LSH functions, new error correcting codes, new data encoders, and train new models for particular speech patterns for the embedding-based scheme.
- Configure attack models to test new and existing options.

We also describe our automated attack suite, which simulates real-world tampering scenarios. Finally, we provide preliminary performance measurements, illustrating each system's performance and resilience to various attacks.

7.1 Implementation Overview

Our implementation is structured as follows:

- **Covertext Generation:**
  - *Watermarking Scheme:* By default, we include GPT-2 and a nanoGPT Shakespeare model [22] for demonstration. Any GPT-style transformer exposing next-token probabilities can be added.
  - *Embedding Scheme:* We provide example scripts using "gpt-4o-mini" for text generation, but the system can interface with any OpenAI API-compatible LLM.
- **Message Encoding & Decoding:**

- *Watermarking Scheme:* Perturbs token distributions (Section 6.1) to embed bits, with a salting mechanism reducing repeated c-gram patterns across multiple messages.
- *Embedding Scheme:* Uses a Locality-Sensitive Hash (LSH) to ensure the text's embedding aligns with target bits (Section 6.2), employing rejection sampling until a suitable candidate is found.

– **Error Correction and Data Encoding:**
  - *Repetition Codes* and *Convolutional Codes* help mitigate lower-than-ideal raw accuracy in the LSH. When stronger error correcting codes (ECC) are used, encoded messages grow in length, balancing accuracy vs. covertext length.
  - Data-encoding utilities allow users to embed text and ciphertext blocks under several encoding schemes.

– **LSH:**
  - *Random Projection LSH* as a baseline.
  - *PCA-based LSH* (Section 7.3), trained on corpora such as Enron emails and Wikipedia articles to provide domain-tailored projections.

– **Attack Simulator:** Automates realistic tampering methods, including n-gram shuffling, synonym substitution, and AI-based paraphrasing. Users can set parameters[3] (e.g., fraction of replaced text and local or global options) to gauge robust message recovery under various threat levels.

This modular design allows new cryptographic primitives, language models, LSH variants, or ECC schemes to be used with minimal changes to the rest of the system.

7.2 Watermarking-Based System

Our watermarking-based system (Section 6.1) encodes bits into covertext by perturbing token probability distributions. Key components include:

– **Token Perturbation:** We implement two sample perturbation functions—one operating on undetectable subtle shifts (similar to and Kirchenbauer et al.'s and Aaronson's watermarking schemes [24,1]) and a "sharp" perturbation that zeros out token probabilities for debugging and testing. Both extract the raw token probabilities from the chosen LLM and apply a keyed pseudorandom function (PRF) to "boost" or "suppress" selected tokens.

– **PRF Integration:** Our default setup uses HMAC-based PRFs, but the design allows any keyed PRF to be provided by the user and an example of adding an AES based option. PRF outputs guide which token probabilities get altered; users can also configure a perturbation strength δ.

– **Length Estimation:** Before encoding, the system calculates required covertext length requirements using empirical statistics (Appendix A) to ensure the hidden message is recoverable at a specified probability ϵ.

– **History Salt:** Each new message is salted with a function of the current conversation state so repeated c-grams across multiple stegotexts do not yield identical perturbations.

[3] The attack system includes three main modes: n-gram shuffle which breaks the text into n-grams and shuffles them; synonym swaps which replaces words with their synonyms; and paraphrase which has an LLM paraphrase the text. These include local and global options, which runs the attack contained within 1 sentence chunks or the entire text respectively. One can also specify the tokenizer, parameters such as n, the proportion of the text that should be attacked, the source/method of synonyms, the model to use for paraphrasing, and set the temperature of the paraphrasing model.

Usage and Customization. Users configure model selection, PRF parameters, c-gram size, and perturbation parameters. The system produces a final stegotext.

7.3 Embedding-Based System

In the embedding-based system (Section 6.2), candidate texts are sampled and then accepted or rejected based on their LSH output. The main features are:

- **Flexible Embedding Models:** Our examples use a "gpt-4o-mini" model for text generation and `text-embedding-3-large` for embedding extraction, though other OpenAI API endpoints can be substituted.
- **Rejection Sampling:** If the text's embedding fails to yield the correct LSH bits, the sample is rejected and a new one is generated.
- **Error Correction:** The built-in repetition and convolutional codes help protect hidden bits from hashing errors. With multi-bit hashes, strong ECC often becomes crucial to maintain acceptable decoding success rates.

Cost Tracking. For the embedding-based system using OpenAI-based text generation, we empirically find the cost to be less than 1¢ per hidden byte. See Section 7.5 for a full analysis.

LSH Design We provide two LSH variants:

- **Random Projection:** A baseline approach that projects embeddings onto random hyperplanes to generate bits.
- **PCA-based Hash:**
  - *Training:* A PCA model is trained on text embeddings from a target domain (e.g., Enron "corporate speech," Wikipedia articles).
  - *Hashing:* Each principal component forms a dimension in the transformed embedding space, thresholded to yield one bit. Combining multiple dimensions produces an l-bit hash.

To measure performance we ran two tests. In the first, we sampled the next message from an initial history 1000 times, paraphrased each sampled message, and compared the accuracy of both functions (they are the same if both messages hash to the same value and different otherwise). The Random Projection had an accuracy of 87.6% and the PCA Hash had an accuracy of 91.6%. In the second, we conintuously sampled 1000 messages using the initial history plus the previous sampled messages as the history, paraphrased each sampled message, and compared the accuracy of both functions. The Random Projection had an accuracy of 77.1% and the PCA Hash had an accuracy of 83.8%[4].

Simulator for Development and Testing A separate simulator tool enables evaluation of different embeddings or the effects of new LSH algorithms by using an oracle. This allows experimentation with minimal API usage, particularly when tuning parameters.

[4] These were particularly long samples, and for shorter messages we observe rates of 93% for the Random Projection Hash and 95-97% for the PCA Hash.